

Storyline

We are in an age in which video games have become commonplace for people of all ages. There are many game systems, each with unique games, allowing users to play whatever is fun for them. However, one thing all video games share is that, when playing an Online game, the user must connect to the game server to play with other online players. For our project, imagine we have just finished creating an online open-world game and must decide how users will join the servers and start playing. This is important because we are planning for this game to be a success, which means the server's capacity will be an issue. While there are currently no plans for expansion, we want to ensure that each user, understands how the game queue will work when entering the game servers.

Players are logging into the server to play. Scheduling in this setting is important since due to the finite amount of players a server can host at a time, there's bound to be more players than the server's able to handle. Therefore, there must be a fair and efficient way to choose which players to let in and which to transfer to another server. For this reason, we have decided that these three options would make the most sense. First, the First Come First Serve algorithm was chosen because this is how many game servers operate today, meaning whatever user joins first gets access first. Second, Shortest Job First was considered, which admits players into the server based on how long it will take the server to fully process user information. It will admit players with the quickest execution time and ignore users who carry a large execution time. Finally, the Priority Scheduling algorithm. This means we are planning on creating a premium service that allows users to cut the line, who enter before non-premium users, getting into the server first. These algorithms highlight the different methods in which we can ensure either happiness and/or profit from users when connecting to the game server.

Algorithms (Non-Preemptive)

- **FCFS**: Players are allowed server access strictly in the order they join the queue. This is simple to implement and easy to understand, but if an early-arriving player has a long load time, every player behind them is blocked and forced to wait. This would cause load times to vary largely between players, creating bottlenecks.
- **SJF**: The server admits players with the fastest connection times first, clearing the queue as fast as possible and maximizing throughput. The downfall is that players with slow internet or large downloads get pushed back indefinitely, causing starvation. Fairness is thrown out with this method, but it will allow the majority of people to quickly gain access to servers.
- **Priority Scheduling**: Priority level is assigned to players based on their account tier. Premium members are placed at the front of the admission queue, leaving free players or new players in the back. In the event of multiple users having the same priority, scheduling will fall back to whoever arrives first. The risk of starvation of free players would cause casual gamers to stray away from the game completely.

Initial Input Design

Process (Unique and Individual Player)	Arrival (Start establishing a connection to the server)	Burst (Time it takes for the server to authenticate the user)	Priority (Premium users who are given access to the front of the line to join the server)
User1 [Free]	0	6	2
User2 [Free]	1	3	2
User3 [Priority]	2	5	1
User4 [Free]	4	1	2
User5 [Priority]	6	4	1

First Come First Serve:

Scheduling Order:

User1 → User2 → User3 → User4 → User5

- **0-6:** User1 (0), User2 (3), User3 (5), User4 (1), User5 (4)
- **6-9:** User2 (0), User3 (5), User 4(1), User5 (4)
- **9-14:** User3 (0), User4 (1), User5 (4)
- **14-15:** User4 (0), User5 (4)
- **15-19:** User5 (0)

Process (Unique User)	Arrival	Burst	CT	WT: TAT-BT	TAT: CT-AT
User1 [Free]	0	6	6 [6+0]	0	6
User2 [Free]	1	3	9 [6+3]	5	8
User3 [Priority]	2	5	14 [9+5]	7	12
User4 [Free]	4	1	15 [14+1]	10	11

User5 [Priority]	6	4	19 [15+4]	9	13
-----------------------	---	---	-----------	---	----

Average WT: $(0+5+7+10+9 = 31/5 = 6.2)$ | **Average TAT:** $(6+8+12+11+13 = 50/5 = 10)$

Shortest Job First:

Scheduling Order:

User1 → User4 → User2 → User5 → User3

- 0-6: User1 (0), User2 (3), User3 (5), User4 (1), User5 (4)
- 6-7: User4 (0), User2 (3), User3 (5), User5 (4)
- 7-10: User2 (0), User3 (5), User5 (4)
- 10-14: User5 (0), User3 (5)
- 14-19: User3 (0)

Process (Unique User)	Arrival	Burst	CT	WT: TAT-BT	TAT: CT-AT
User1 [Free]	0	6	6 [6+0]	0	6
User2 [Free]	1	3	10 [7+3]	6	9
User3 [Priority]	2	5	19 [14+5]	12	17
User4 [Free]	4	1	7 [6+1]	2	3
User5 [Priority]	6	4	14 [10+4]	4	8

Average WT: $[(0+6+12+2+4)/5] = 4.8$ | **Average TAT:** $[(6+9+17+3+8)/5] = 8.6$

Priority Scheduling:

Scheduling Order: User1 → User3 → User5 → User2 → User4

- 0-6: User1 (0), User2 (B:3, P: 2), User3 (B:5, P:1), User4 (B:1, P:2), User5 (B:4, P:1)
- 6-11: User3 (0), User2 (B:3, P: 2), User4 (B:1, P:2), User5 (B:4, P:1)
- 11-15: User5 (0), User2 (B:3, P: 2), User4 (B:1, P:2)
- 15-18: User2 (0), User4 (B:1, P:2)
- 18-19: User4 (0)

Process (Unique User)	Arrival	Burst	Priority	CT	WT: TAT-BT	TAT: CT-AT
User1 [Free]	0	6	2	6 [6+0]	0	6
User2 [Free]	1	3	2	18 [15+3]	14	17
User3 [Priority]	2	5	1	11 [6+5]	4	9
User4 [Free]	4	1	2	19 [18+1]	14	15
User5 [Priority]	6	4	1	15 [11+4]	5	9

Average WT: $[(0+14+4+11+5)/5] = 6.8$ | **Average TAT:** $[(6+17+9+15+9)/5] = 11.2$

Group Info:

Ap Calderon: Main Coder

Calista Ruiz: Comparison/Analysis, Algorithms, Presentation

Erick Rayas: Documentation, Presentation

Linh Tran: Input/Output Design, Presentation

Raul Munoz: Input/Output Design, Presentation, Storyline, Supplemental Coder[if needed]

Implementation

Language: C/Python

Packages/Libraries: None at the moment.

Meetings: Every Friday to communicate progress with code implementation/handling concerns.